Midterm Laboratory Activity: Portfolio CRUD with Validation

Converting Portfolio to Laravel + Inertia.js + React

Item	Details
Course	IT ELEC01 – Advanced Web Development
Instructor	Giobet T. Aban
Grouping	Individual or Pair (2)
Deadline	November 3, 2026
Submission	GitHub repository link + Video Demonstration link

Overview

In the previous activity, you built your personal portfolio website. In this activity, you will **convert your portfolio into a Laravel + Inertia.js + React application** and make **one section of it dynamic** by building a complete **CRUD (Create, Read, Update, Delete)** feature with proper **validation**.

Instead of hard-coding the content of that section, its data will now be stored in a database and managed through forms. Visitors still see your portfolio as before, but you can now add, edit, and remove content without touching the code.

Building the CRUD is only part of the task. You must also be able to **explain how it works and why you built it that way**.

Learning Outcomes

By the end of this activity, you should be able to:

1. Convert a static website into a Laravel + Inertia.js + React application.
2. Build a working CRUD feature using a migration, an Eloquent model, a resource controller, routes, and Inertia React pages.
3. Design server-side validation rules and display validation errors properly in the React UI.
4. Explain the request flow of a Laravel + Inertia application from route to rendered page.
5. Justify the design decisions behind your implementation.

Part 1 - Convert Your Portfolio

1. Create a new Laravel project with Inertia.js and React (use the starter setup discussed in class).
2. Move your existing portfolio into the project as React pages and components. Break repeated parts into components (e.g., Navbar, Footer, SectionTitle, Card).
3. Keep your original design. Use Tailwind CSS for styling.
4. Your public portfolio page must still display correctly after conversion.

Part 2 - Choose ONE Section to Make Dynamic

Choose **one** section of your portfolio to convert into a CRUD feature. Examples:

Section	Possible Fields
Skills	name, category, proficiency level (1–100), icon, display order
Projects	title, description, tech stack, project URL, repository URL, image, date completed
Gallery	title, caption, image, category, date taken
Experience / Education	title, organization, start date, end date, description

Section	Possible Fields
Certificates	title, issuer, date issued, credential URL, image
Testimonials	name, position, message, photo

You may choose another section as long as it has at least **five meaningful fields** (excluding id and timestamps).

In your documentation, answer the following:

- Which section did you choose, and **why** this section? Why does it make sense to manage this content through a database instead of hard-coding it?
- What are your fields? For each field, give its data type and explain **why it is needed**.
- What are at least **five rules** your data must follow? (Example: "A skill's proficiency must be between 1 and 100," "A project URL must be a valid URL," "End date cannot be earlier than start date.") These rules will become your validation rules.

Part 3 - Build the CRUD

Your CRUD feature must meet these **minimum technical requirements**:

1. **Migration:** Creates the table for your chosen section with at least **5 meaningful fields**.
2. **Model:** An Eloquent model with a properly defined $fillable property.
3. **Controller:** A **separate resource controller** for your section (e.g., ProjectController, SkillController) with index, create, store, edit, update, and destroy methods.
4. **Routes:** Registered in routes/web.php using Route::resource() (or explicitly defined routes that you can justify).
5. **Validation:** At least **five different validation rules** across your fields, including:
   a. required
   b. a length or size rule (max, min, between)
   c. a type or format rule (numeric, integer, date, url, email, etc.)
   d. a unique rule that correctly **ignores the current record on update**, if applicable to your fields
   e. for image uploads: image, mimes, and max file size
   f. at least one **custom error message**
6. **React Pages:** At minimum Index, Create, and Edit pages inside resources/js/Pages/[YourSection]/.
7. **Components:** At least one reusable component (e.g., a shared form used by both Create and Edit, or an input with an error message).
8. **Error Display:** Validation errors must appear beside the related field, and the values typed by the user must not be lost when validation fails.
9. **Feedback:** A success message after create, update, and delete.
10. **Delete Confirmation:** A confirmation step before a record is deleted.
11. **Public Display:** The records you manage must appear in that section of your **public portfolio page**.

Bonus (optional): search or filter, pagination, drag-to-reorder or display order, a Show page, or login protection for the management pages.

Part 4 - Test Your Validation

Prove that your validation works. Create a **Validation Test Table** with at least **six test cases**, for example:

- Submitting the form with all fields empty
- Exceeding the maximum length of a text field
- Entering the wrong data type or format (e.g., letters in a number field, invalid URL)
- Entering a duplicate value in a field that must be unique
- Updating a record without changing its unique field (it should still save)
- Uploading a file that is not an image, or an image that is too large

#	Test Case	Input Used	Expected Result	Actual Result	Pass / Fail
1	Empty form	All fields blank	Error message under each required field	[what happened]	[ ]

If a test fails, fix it and write down what you changed.

Part 5 - Video Demonstration

Record a **video demonstration** explaining the CRUD operation of your portfolio. Screen recording with your **voice** is required; showing your face is optional but encouraged.

Your video must cover the following:

Segment	Content
1. Introduction	Your name, your portfolio, the section you chose, and why you chose it
2. Folder Structure	Show where your migration, model, controller, routes, pages, and components are located
3. Code Explanation	Walk through the flow: Route → Controller → Validation → Model → Inertia → React Page. Explain each part in your own words
4. CRUD Demo	Create, view, edit, and delete a record. Show the result on your public portfolio page
5. Validation Demo	Deliberately trigger validation errors from your test table and show how the errors are displayed
6. Live Change	Without cutting the video, add one new field with a validation rule (migration, model, validation, form input) and show it working

Rules for the video:

- Upload to Google Drive or YouTube (unlisted) and make sure the link is viewable.

In your video, you must be able to answer questions like these:

- Why did you create a separate controller for your section instead of writing everything in web.php?
- What routes does Route::resource() create, and which ones are you using?
- What would happen if you removed $fillable from your model?
- Why must validation happen on the server (Laravel) and not only in React?
- How do validation errors from Laravel reach your React page without a full page reload?
- Why did you create a reusable component instead of repeating the same code?

Part 6 - Documentation

Submit one PDF containing the following sections:

1. **Title Page** - Activity title, your name, course, section, date.
2. **Section Chosen and Justification** - Your answers in Part 2.
3. **Screenshots of All Pages** - Label each figure:
    a. Public portfolio page showing the dynamic section
    b. Index page (list of records)
    c. Create page
    d. Edit page
    e. Delete confirmation
    f. Success message
    g. Validation errors displayed on the form
4. **Folder Structure (screenshots)** - Show the files you created or modified.
    Example (for a Projects section):
    a. app/Http/Controllers/ProjectController.php
    b. app/Models/Project.php
    c. database/migrations/xxxx_xx_xx_create_projects_table.php
    d. routes/web.php
    e. resources/js/Pages/Projects/
        i. Index.jsx
        ii. Create.jsx
        iii. Edit.jsx
    f. resources/js/Pages/Portfolio.jsx
    g. resources/js/Components/
        i. ProjectForm.jsx
        ii. Navbar.jsx
5. **Source Code** - Include the **complete code** of the following, each with a short explanation (2–3 sentences) in your own words:
    a. Migration
    b. Model
    c. Controller (including validation rules and custom messages)
    d. Routes (web.php)
    e. React Pages (Index, Create, Edit, and the public portfolio page)
    f. Components
6. **Design Decisions** - Answer briefly:
    a. Why did you use a separate resource controller?
    b. Why did you choose your validation rules? Where did you place them and why?
    c. How did you organize your React pages and components, and why?
7. **Validation Test Table** - Your Part 4 table.
8. **Reflection** - Answer in one paragraph each:
    a. What was the hardest part of converting your portfolio, and how did you solve it?
    b. What did you learn about validation that you did not know before?
    c. If you had to add a second dynamic section, what would you do differently?
9. **Links or Source codes** - GitHub repository and video demonstration.

Grading Rubric (100 points)

Criteria	Exemplary	Proficient	Developing	Beginning
Portfolio Conversion (10 pts)	Portfolio fully converted with original design intact and well-organized components (10)	Converted with minor layout issues (7–9)	Partially converted (4–6)	Not converted or not working (0–3)
Functional CRUD (20 pts)	All CRUD operations work; records appear on the public page; success messages and delete confirmation present (20)	All CRUD works with minor issues (15–19)	Some operations missing or buggy (8–14)	CRUD mostly not working (0–7)
Validation (20 pts)	5+ rule types including custom messages; errors shown per field; input retained; 6+ test cases documented (20)	Rules complete; test table mostly complete (15–19)	Few rules; incomplete testing (8–14)	Little or no validation (0–7)
Video Demonstration (30 pts)	Clear explanation of the full flow in own words; all segments covered; live change succeeds; questions answered confidently (30)	Mostly clear; minor gaps; live change mostly works (22–29)	Reads from script; missing segments (12–21)	Cannot explain own code; no live change (0–11)
Documentation (20 pts)	All screenshots and complete code included with clear explanations; thoughtful design decisions and reflection (20)	Complete with minor gaps (15–19)	Several sections missing; generic explanations (8–14)	Mostly missing (0–7)

Submission Checklist

- [ ] GitHub repository is accessible and includes a README with setup instructions
- [ ] Documentation PDF includes all 9 sections
- [ ] Screenshots of all required pages are included and labeled
- [ ] Complete code of migration, model, controller, routes, pages, and components is included
- [ ] Video, link is viewable or unlisted.

Giobet T. Aban
Instructor, BSIT